Програмний проєкт 2. Варіант 1: std::all_of — дослідження швидкодії

Постановка. Компілятор MSVC 1944, C++20, політики виконання через TBB; апаратних потоків: 8. Послідовності: випадкові int з рівномірного розподілу [1; 10^6] (mt19937, зерно фіксоване), довжини 10^5, 10^6, 10^7, збережені у seq_N.txt (формат: перший рядок N, другий — N чисел) і однакові для всіх режимів. Усі елементи задовольняють предикат, тож all_of=true і виконується повний прохід (найгірший випадок). Предикати: швидкий x>0 та повільний exp(log1p(x))>0.5. Вимірювалась тривалість одного виклику алгоритму (без читання файлу); у таблицях — медіана 31/15/7 запусків для N=10^5/10^6/10^7, мс. Власний алгоритм ділить вибірку на K майже рівних частин, кожну обробляє окремий потік (std::all_of), результати зводить std::all_of. Правильність перевірено самотестом (2000 випадкових вхідних, у т. ч. з порушеннями).

1–2. Бібліотечний all_of: без політики та з політиками, -O3 та -O0 (мс)

		-O3 (макс. оптимізація), мс					-O0 (без оптимізації), мс				
N	**Предикат**	**no policy**	**seq**	**par**	**par_unseq**	**unseq**	**no policy**	**seq**	**par**	**par_unseq**	**unseq**
100 000	x>0	0.036	0.036	0.038	0.032	0.036	0.666	1.84	1.95	1.79	2.00
100 000	exp(log1p x)	1.98	1.82	0.533	0.515	1.99	2.96	5.02	4.32	4.05	3.99
1 000 000	x>0	0.502	0.815	0.123	0.135	0.367	6.65	19.2	19.0	17.4	21.3
1 000 000	exp(log1p x)	20.9	20.2	4.87	4.62	26.8	30.6	39.2	41.8	41.7	41.4
10 000 000	x>0	5.83	5.42	2.54	3.09	4.93	65.8	192	185	170	207
10 000 000	exp(log1p x)	236	257	49.3	51.5	285	313	431	419	437	423

3. Власний паралельний алгоритм: час (мс, -O3) залежно від K

N	**Предикат**	**1**	**2**	**3**	**4**	**6**	**8**	**12**	**16**	**24**	**32**	**48**	**64**	**128**	**256**	**512**	**1024**
100 000	x>0	0.103	0.137	0.164	0.203	0.272	0.352	0.505	0.653	0.957	1.25	1.87	2.52	5.10	10.5	21.0	44.9
100 000	exp(log1p x)	2.18	1.43	1.29	1.09	0.850	0.805	0.849	0.936	1.15	1.28	1.81	2.98	5.53	10.9	21.9	43.5
1 000 000	x>0	0.678	0.662	0.315	0.288	0.374	0.450	0.568	0.743	1.10	1.37	1.94	2.66	5.68	10.2	20.6	41.1
1 000 000	exp(log1p x)	19.9	11.8	10.8	8.84	7.05	6.13	6.49	6.50	7.86	6.82	7.11	7.04	8.73	13.1	23.7	42.5
10 000 000	x>0	4.86	2.84	2.81	2.79	2.38	2.58	2.41	2.38	2.59	2.62	2.69	3.20	5.49	10.5	21.2	41.8
10 000 000	exp(log1p x)	284	150	112	92.6	64.9	77.2	68.1	70.9	55.1	64.8	188	72.2	68.9	98.7	113	110

Найкраще K*, прискорення відносно послідовного all_of, лінійна апроксимація t = a + b·K при K ≥ 4·потоків, найкраще K при -O0

N	**Предикат**	**K*(O3)**	**t(K*), мс**	**K*/потоків**	**прискор. до seq**	**b, мс/потік**	**R^2**	**K*(O0)**
100 000	x>0	1	0.103	0.12	0.36×	0.0438	0.999	1
100 000	exp(log1p x)	8	0.805	1.00	2.46×	0.0425	1.000	2
1 000 000	x>0	4	0.288	0.50	1.28×	0.0399	1.000	1
1 000 000	exp(log1p x)	8	6.13	1.00	3.41×	0.0366	0.997	8
10 000 000	x>0	6	2.38	0.75	2.18×	0.0400	0.999	16
10 000 000	exp(log1p x)	24	55.1	3.00	5.68×	0.0119	0.010	256

Висновки

Політики. seq і відсутність політики відрізняються не більше ніж на 62% (це той самий послідовний прохід). Для N=10 000 000 політика par дає прискорення відносно «no policy» 2.30× (x>0) і 4.80× (exp/log); par_unseq — 1.89× і 4.59×. Навіть для швидкого предиката виграш помітний. Для повільного предиката паралелізм виправданий — обчислення домінують над накладними витратами.

Рівень оптимізації. -O3 швидший за -O0 у 11.3× для x>0 (інлайнінг предиката та векторизація) і лише у 1.3× для exp/log, де час домінують виклики libm, які оптимізація не прискорює.

Найкраще K. N=100 000, x>0: K*=1 (0.12·потоків); N=100 000, exp(log1p x): K*=8 (1.00·потоків); N=1 000 000, x>0: K*=4 (0.50·потоків); N=1 000 000, exp(log1p x): K*=8 (1.00·потоків); N=10 000 000, x>0: K*=6 (0.75·потоків); N=10 000 000, exp(log1p x): K*=24 (3.00·потоків). Кількість апаратних потоків: 8. Для найважчого випадку K*/потоків = 3.00, тобто відхиляється від кількості потоків (для швидких предикатів виграш маскується накладними витратами).

Закон зростання за K □ потоків. Для K ≥ 4·потоків час добре описується лінійною залежністю t = a + b·K (R^2 від 0.010, b від 0.0119 до 0.0438 мс/потік): корисна робота N/K на потік зменшується, але вартість створення та приєднання (join) одного потоку стала, тому сумарні накладні витрати □ K, а загальний час зростає лінійно — O(K).